



# RAGHU INSTITUTE OF TECHNOLOGY

(Autonomous)

*(Approved by AICTE, New Delhi, Accredited by NBA & Re Accredited by  
NAAC with 'A+' grade & Permanently Affiliated to JNTU-GV Vizianagaram)*

## CODE REVIEW SYSTEM USING Generative AI

Under the esteemed guidance of  
**Dr. P. APPALA NAIDU**  
(Professor & HoD)

Presented by:  
**H. PRAVALIKA(213J1A0565)**  
**G. SUNIL KUMAR(213J1A0551)**  
**A. TARUNI(213J1A0501)**  
**G. PRENITH SAI(213J1A0563)**

# ABSTRACT

The **Code Review System using Generative AI** is an automated solution designed to enhance software development by providing intelligent code reviews. Traditional code review methods require significant manual effort, making the process time-consuming and prone to human error. This project leverages **Google's Gemini-Pro model** and **LangGraph** to automate code analysis, feedback generation, and optimization. The system follows an iterative workflow that includes **code evaluation, refinement, rating, and optimization** to ensure adherence to industry best practices. Using **natural language processing (NLP) and AI-powered insights**, the system provides constructive feedback, suggests code improvements, and enhances overall code quality. By integrating **Streamlit for an interactive UI**, the system offers a seamless user experience for developers. This AI-assisted approach significantly reduces manual effort, improves developer productivity, and ensures the delivery of optimized and maintainable code, making it a valuable tool for modern software development practices.

# LITERATURE SURVEY

AI Code Review: Advancing code quality with AI-enhanced reviews

Link: <https://www.sciencedirect.com/science/article/pii/S2352711024000487>

| Algorithms Used                                                                                                                            | Working                                                                                                                                                                                                                                                              | Drawback                                                                                                                                                                                                                                                                       | Datasets Used                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <ol style="list-style-type: none"><li>1. GPT-3.5-based NLP engine.</li><li>2. Neural networks for syntax and semantic analysis..</li></ol> | <ol style="list-style-type: none"><li>1. Code snippets are parsed for syntax/semantic discrepancies.</li><li>2. Suggestions are generated using a pre-trained language model.</li><li>3. Focused on integrating with IntelliJ IDEA for real-time analysis.</li></ol> | <ol style="list-style-type: none"><li>1. Reliance on GPT-3.5 limits domain-specific accuracy.</li><li>2. Limited support for programming languages other than those explicitly configured.</li><li>3. Potential overfitting due to fine-tuning on specific datasets.</li></ol> | <ol style="list-style-type: none"><li>1. Code repositories analyzed via IntelliJ IDEA plugin, leveraging GPT-3.5 for assessment.</li></ol> |

# Architecture Diagram

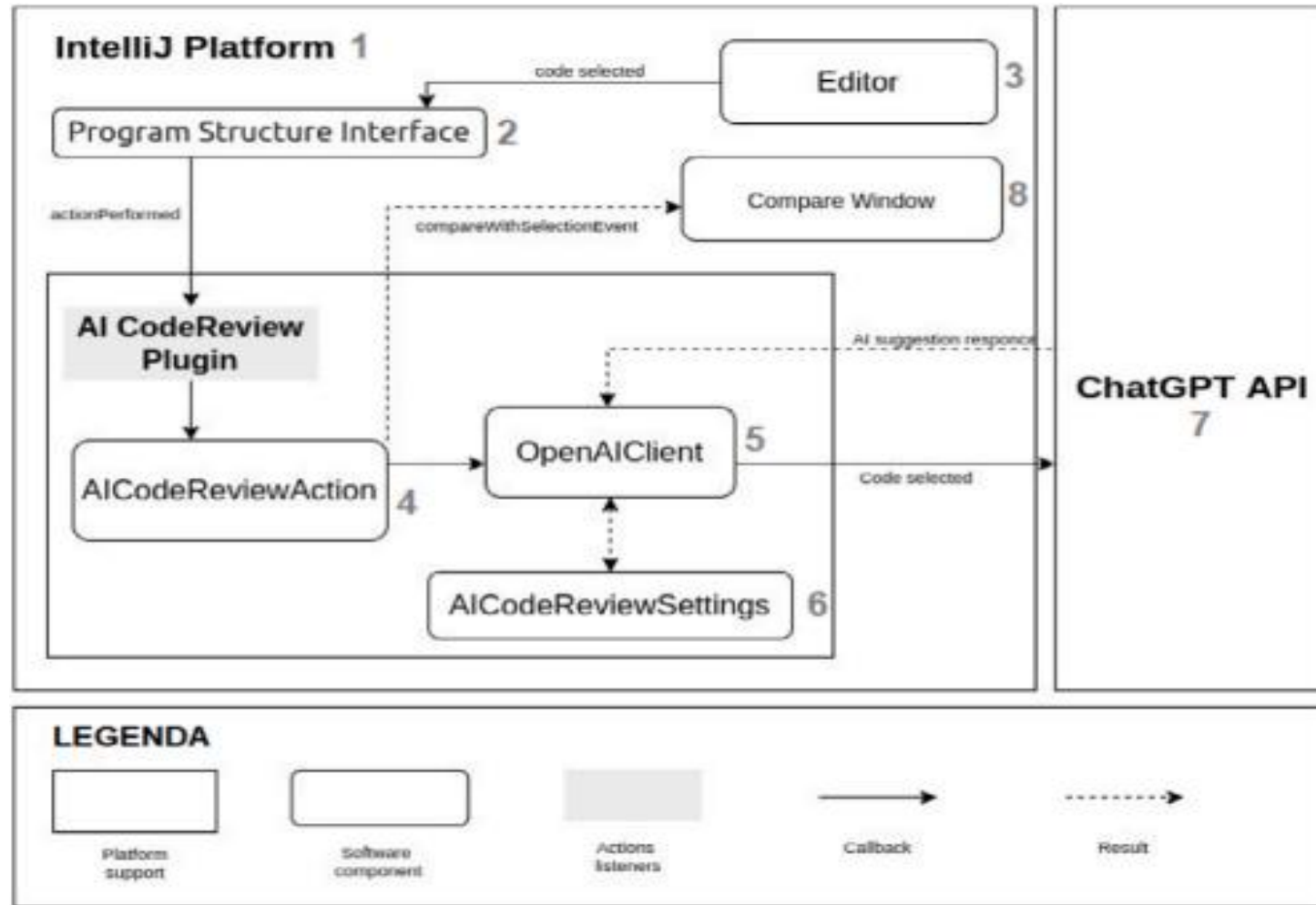


Fig-1: AI Code Review

## Enhancing Software Quality through AI - Assisted Code Review

Link: <https://www.ijsr.net/archive/v12i2/SR24716230727.pdf>

| Algorithms Used                                                                                                                                                                           | Working                                                                                                                                                                                                     | Drawback                                                                                                                                                                                                                                                                          | Datasets Used                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ol style="list-style-type: none"><li>1. Rule-based systems combined with LLMs for best practice enforcement.</li><li>2. Hybrid approaches using style guide-specific datasets.</li></ol> | <ol style="list-style-type: none"><li>1. Verifies adherence to best practices and assists reviewers with contextual feedback.</li><li>2. Optimized for Google's large-scale industrial codebases.</li></ol> | <ol style="list-style-type: none"><li>1. Scalability issues in non-Google environments.</li><li>2. Requires extensive customization for non-standard best practices.</li><li>3. Partial automation; still heavily reliant on human reviewers for functional validation.</li></ol> | <ol style="list-style-type: none"><li>1. Historical code review data from AWS Cloud Infrastructure Development.</li><li>2. Includes open-source and enterprise-level datasets for validation.</li></ol> |

## Architecture Diagram

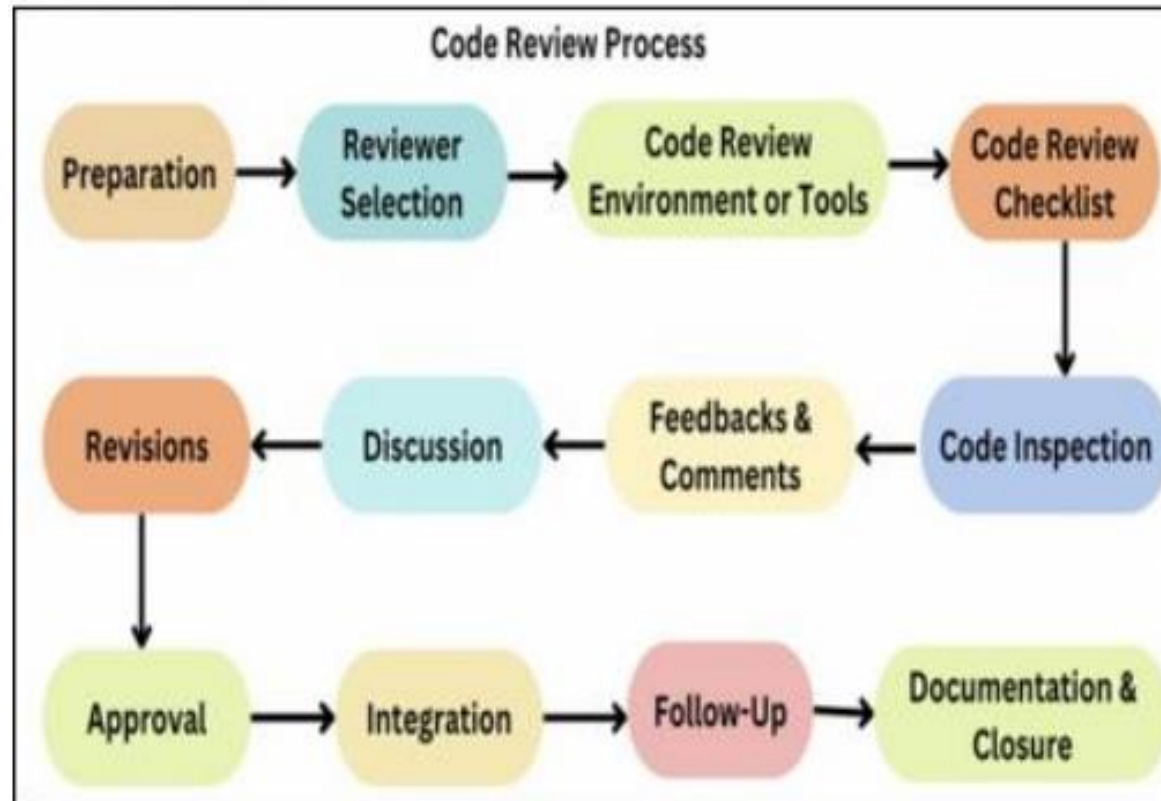


Fig-2: Enhancing Software Quality through AI - Assisted Code Review

# A GPT-based Code Review System for Programming Language Learning

Link: <https://arxiv.org/abs/2407.04722v1>

| Algorithms Used                                                                                                                                                    | Working                                                                                                                                                                                                                                  | Drawback                                                                                                                                                                                                                                                      | Datasets Used                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <ol style="list-style-type: none"><li>1. GPT-4 for personalized code reviews.</li><li>2. Dataset-enhanced prompts for accuracy in educational scenarios.</li></ol> | <ol style="list-style-type: none"><li>1. Collects learner-submitted code and highlights areas for improvement.</li><li>2. Reduces potential AI-assist cheating by focusing on submitted code rather than generating solutions.</li></ol> | <ol style="list-style-type: none"><li>1. Context limitations due to dataset dependency.</li><li>2. Slower in larger educational environments due to scalability issues.</li><li>3. Vulnerable to false negatives for non-standard code submissions.</li></ol> | <ol style="list-style-type: none"><li>1. Training data includes public repositories and educational code snippets tailored for teaching.</li></ol> |

# Architecture Diagram

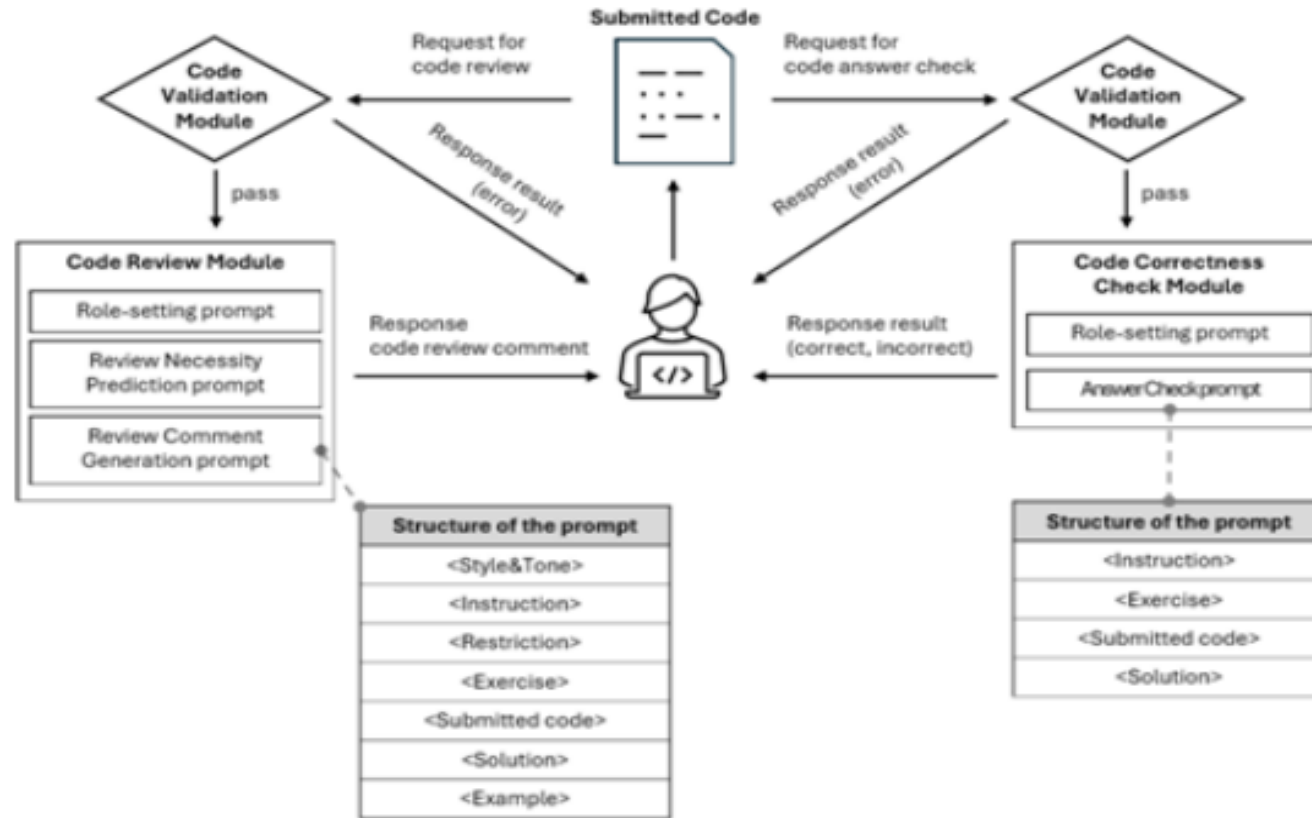


Fig-3: A GPT-based Code Review System for Programming Language Learning

# AI-Assisted Assessment of Coding Practices in Modern Code Review

Link: <https://arxiv.org/abs/2405.13565>

| Algorithms Used                                                                                                                                            | Working                                                                                                                                                                 | Drawback                                                                                                                                                                                                                                    | Datasets Used                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>1. Large Language Models (LLMs) for natural language feedback.</div><div>2. Static analysis integration for auto-comment generation.</div></div> | <div><div>1. Generates automated comments based on best practice violations.</div><div>2. Enables reviewers to focus on logic rather than style violations.</div></div> | <div><div>1. High false positive rates for nuanced style issues.</div><div>2. Limited adaptability to evolving best practices in codebases.</div><div>3. Reviewer reliance on automated suggestions may reduce manual scrutiny.</div></div> | <div><div>1. Code submissions from diverse programming languages (C++, Java, Python, Go).</div><div>2. Likely includes open-source datasets and repositories.</div></div> |

## Architecture Diagram

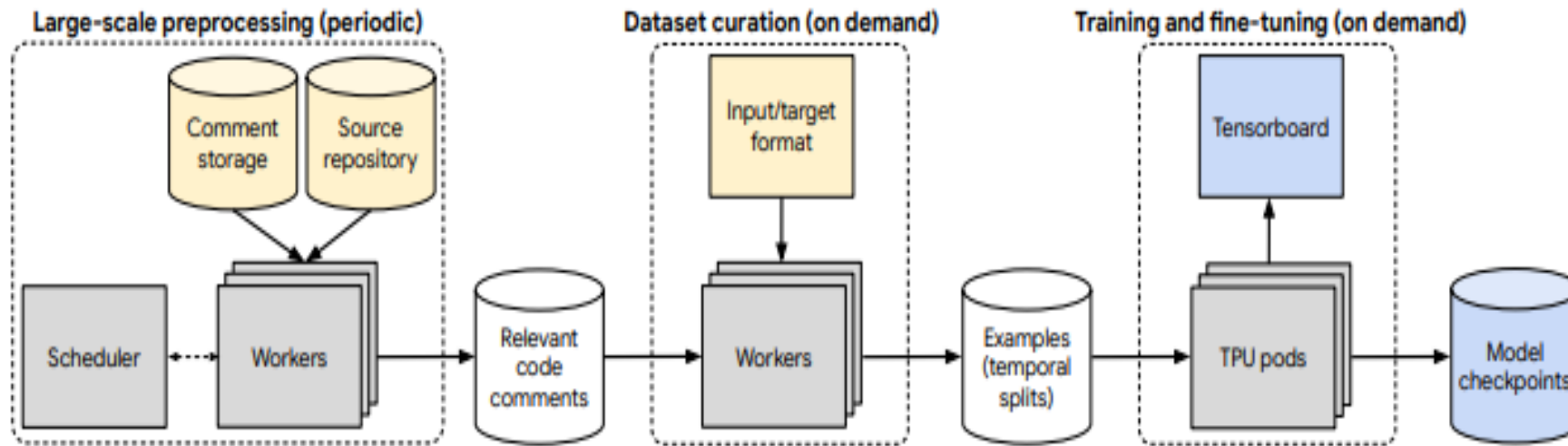


Fig-4: AI-Assisted Assessment of Coding Practices in Modern Code Review

# AI-powered Code Review with LLMs

Link: <https://arxiv.org/abs/2404.18496>

| Algorithms Used                                                                                                                                                | Working                                                                                                                                                                                               | Drawback                                                                                                                                                                                                                                                                                         | Datasets Used                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>1.</div><div>Transformer-based models for contextual analysis.</div><div>2.</div><div>Multi-level embeddings for semantic understanding.</div></div> | <div><div>1.</div><div>Detects code smells, predicts potential bugs, and suggests optimizations</div><div>2.</div><div>Enhances developer education through detailed feedback mechanisms.</div></div> | <div><div>1.</div><div>Over-reliance on pre-trained datasets can lead to biased recommendations.</div><div>2.</div><div>Limited empirical validation against manual reviews in industrial settings.</div><div>3.</div><div>Poor performance in detecting deeper architectural flaws.</div></div> | <div><div>1.</div><div>Large code repositories from GitHub, bug reports, and documentation of best practices.</div><div>2.</div><div>Dataset sourced via GitHub REST API for public repositories and historical pull requests.</div></div> |

# Optimizing AI-Assisted Code Generation: Enhancing Security, Efficiency, and Accessibility in Software Development

Link: <https://arxiv.org/abs/2412.10953v1>

| Algorithms Used                                                                                                                                                               | Working                                                                                                                                                                                                                | Drawback                                                                                                                                                                                                                                                                                | Datasets Used                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ol style="list-style-type: none"><li>1. Encoder-Decoder Transformer models (e.g., GPT-3, Codex, AlphaCode).</li><li>2. Static analysis tools integrated with LLMs.</li></ol> | <ol style="list-style-type: none"><li>1. AI generates, evaluates, and optimizes code snippets for security and efficiency.</li><li>2. Incorporates feedback loops to improve successive generations of code.</li></ol> | <ol style="list-style-type: none"><li>1. Generated code may contain logical errors despite syntactical correctness.</li><li>2. Struggles with domain-specific languages not included in training data.</li><li>3. High computation cost and dependency on GPU infrastructure.</li></ol> | <ol style="list-style-type: none"><li>1. Pre-trained on extensive repositories like GitHub and datasets used by LLMs (e.g., OpenAI Codex, AlphaCode).</li></ol> |

## Architecture Diagram

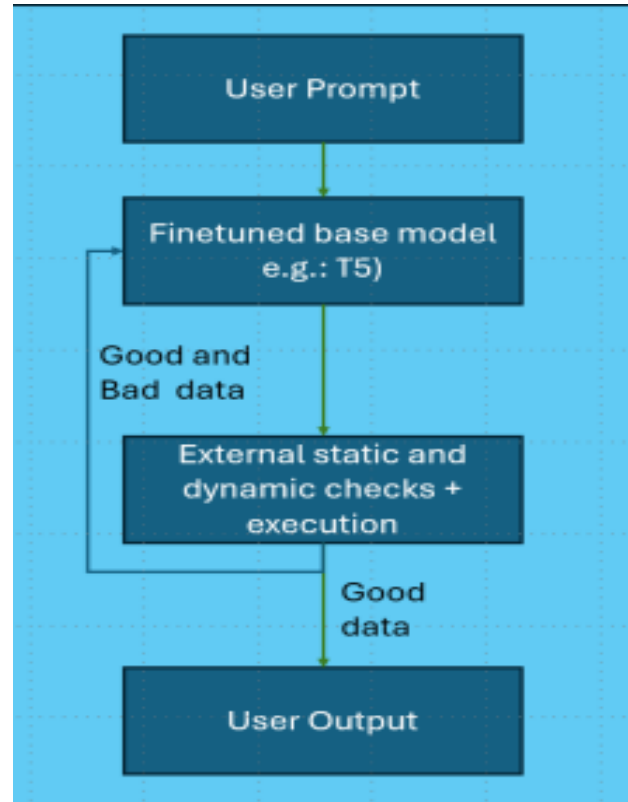


Fig-5: Optimizing AI-Assisted Code Generation: Enhancing Security, Efficiency, and Accessibility in Software Development

# EXISTING SYSTEM

The existing AI-assisted code review systems leverage techniques such as **Abstract Syntax Tree (AST) analysis, static code analysis, heuristic-based approaches, and transformer-based models like GPT-3.5 and Codex** to evaluate code quality. While these tools assist in identifying syntax errors, code smells, and best practices, they suffer from **high false positives, poor adaptability to domain-specific languages, scalability issues, and a lack of deep architectural understanding**. Additionally, reliance on **predefined rules and pre-trained datasets** leads to biased recommendations and inefficiencies in handling evolving coding standards. Furthermore, these systems **require significant computational resources**, making them expensive and slower in large-scale deployments.

# PROPOSED SYSTEM

Our proposed **Code Review System using Generative AI** enhances traditional code review by integrating **Google's Gemini-Pro model, LangGraph for workflow automation, and Streamlit for an interactive UI**, ensuring a **comprehensive, accurate, and scalable** review process. Unlike existing systems, it dynamically adapts to evolving coding practices using **context-aware prompts and iterative feedback loops**. The system efficiently analyzes source code, generates **human-like feedback**, suggests optimized improvements, and evaluates skill proficiency based on review iterations. We utilize **pre-trained datasets and domain-specific benchmarks** to enhance accuracy and reduce false positives. By addressing limitations in **bias, computational efficiency, and adaptability**, our system significantly improves **accuracy, efficiency, and usability**, making it a robust solution for AI-driven code evaluation.

How our system overcome drawbacks of the existing system?

| Drawbacks of Existing Systems                                                     | How Our System Overcomes Them                                                                                  |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Generated code may contain logical errors despite syntactical correctness.        | Implements multi-step iterative reviews to refine logic, ensuring both syntactical and logical correctness.    |
| Struggles with domain-specific languages not included in training data.           | Allows customization for domain-specific languages by integrating user-defined rules and fine-tuning models.   |
| High computation cost and dependency on GPU infrastructure.                       | Optimized architecture reduces computation overhead; can run on cloud-based or local environments efficiently. |
| Potential overfitting due to fine-tuning on specific datasets.                    | Utilizes a diverse range of prompts and iterative validation to prevent overfitting to specific code patterns. |
| Limited support for programming languages other than those explicitly configured. | Supports multiple programming languages with extensibility for future language additions.                      |
| Reliance on GPT-3.5 limits domain-specific accuracy.                              | Uses <b>Google's Gemini-Pro</b> , a more advanced model optimized for technical and code-specific reasoning.   |

# IMPLEMENTATION OF OUR PROJECT

## **Technology Stack Used:**

- **Programming Language:** Python
- **Frontend:** Streamlit (for an interactive UI)
- **Backend:** LangGraph (for workflow automation)
- **AI Model:** Google Gemini Pro via langchain\_google\_genai
- **Version Control:** Git & GitHub

## **Key Features Implemented:**

- **Multi-Step Review Process** (AI-driven code refinement)
- **Automated Skill Rating System** (evaluates coding proficiency)
- **Side-by-Side Code Comparison** (original vs. optimized)
- **Interactive Web UI** (easy to use, real-time feedback)

# CODE SNIPPETS

## Initialization of AI Model (Google Gemini)

```
import google.generativeai as genai
from langchain_google_genai import ChatGoogleGenerativeAI

genai.configure(api_key=GOOGLE_API_KEY)
ai_model = ChatGoogleGenerativeAI(model="gemini-pro",
google_api_key=GOOGLE_API_KEY)
```

## Generate AI Response for Prompts

```
def generate_response(prompt):
    return ai_model.invoke(prompt).content
```

## Define Code Review State

```
from typing import Dict, TypedDict, Optional

class ReviewState(TypedDict):
    feedback_notes: Optional[str] = None
    review_history: Optional[str] = None
    source_code: Optional[str] = None
    programming_language: Optional[str] = None
    skill_rating: Optional[str] = None
    iteration_count: Optional[int] = None
    comparison_result: Optional[str] = None
    original_code: Optional[str] = None
    improved_code: Optional[str] = None
```

## Create StateGraph Workflow

```
from langgraph.graph import StateGraph, END

graph = StateGraph(ReviewState)
```

## Define Review & Improvement Prompts

```
review_prompt = "You are a code reviewer proficient in {}. Analyze the given code and identify potential issues.\nCode:\n {}"  
rewrite_prompt = "You are a developer skilled in {}. Improve the given code by implementing these recommendations:\n {} \n Code:\n {}"  
optimization_prompt = "Enhance the following code for better efficiency and readability while preserving functionality.\nCode:\n {}"
```

## Define Optimization & Finalization Functions

```
def optimize_code(state):  
    improved_code = generate_response(optimization_prompt.format(state['source_code']))  
    return {'improved_code': improved_code}  
  
def finalize_review(state):  
    skill_rating = generate_response(rating_prompt.format(state['review_history']))  
    return {'skill_rating': skill_rating, 'comparison_result': comparison_result, 'improved_code': improved_code}
```

# Streamlit UI for User Interaction

```
import streamlit as st

st.title("🔍 Code Review System using Generative AI")
st.sidebar.header("Project Settings")

programming_language = st.sidebar.selectbox("Select Programming Language", ["C", "Python", "Java", "C++", "JavaScript"])
source_code = st.text_area("Paste Your Code Here", height=250)
if st.button("Start Review Process"):
    with st.spinner("Reviewing code..."):
        st.subheader("📄 Review History")
        st.text_area("Review Process", process['review_history'], height=250)
        st.subheader("📁 Original Code")
        st.code(process['original_code'], language=programming_language.lower())
        st.subheader("🚀 Optimized Code")
        st.code(process['improved_code'], language=programming_language.lower())
        st.subheader("★ Rating")
        st.write(process['skill_rating'])
```

# RESULTS

Share ☆ ✎ ↺ ⋮

## Project Settings

Select Programming Language

C



## Code Review System using Generative AI

Paste Your Code Here

Start Review Process

< Manage app

## Project Settings

Select Programming Language

Java



# Code Review System using Generative AI

Paste Your Code Here

```
public class SwapNumber{  
    public static void main(String args[]){  
        int a=10,b=20,temp;  
        temp=a;  
        a=b;  
        b=temp;  
        System.out.println("a= "+a+"b= "+b);  
    }  
}
```

Start Review Process

 Reviewing code...

[Manage app](#)

## Project Settings

Select Programming Language

Java



Share ☆ ✎ ↺ ⋮

## Review History

Review Process

```
System.out.println("a= "+a+"b= "+b);
}
}
```

REVIEWER:

This Java code swaps two integer variables, but it could be improved in several ways:

**Naming:** The class name `SwapNumber` is slightly awkward. `SwapNumbers` or even a more descriptive name like `NumberSwapper` would be better. Variable names `a` and `b` are too generic. Using `firstNumber` and `secondNumber` would significantly improve readability.

**Whitespace and Formatting:** The code is a bit cramped. Adding more whitespace around operators (`=`, `+`, `;`) and using consistent indentation would make it much easier to read. A block

## Original Code

```
public class SwapNumber{
public static void main(String args[]){
int a=10,b=20,temp;
temp=a;
a=b;
b=temp;
System.out.println("a= "+a+"b= "+b);
}
```

< Manage app

## Project Settings

Select Programming Language

Java



Share



## Optimized Code

**Several** improvements can be made to the code for efficiency and readability. 

**\*\*1. Using** a temporary variable (improved readability):\*\*

**This** version improves readability by adding more descriptive variable names and fo

```
```java
public class SwapNumbers {
    public static void main(String[] args) {
        int firstNumber = 10;
        int secondNumber = 20;
        int temporaryHolder;

        temporaryHolder = firstNumber;
        firstNumber = secondNumber;
        secondNumber = temporaryHolder;

        System.out.println("First number: " + firstNumber + ", Second number: " +
    }
}
```

< Manage app

## Project Settings

Select Programming Language

Java



Share



## Specialization

Java

## Rating

I'd rate the coder's proficiency a **6/10**.

Reason: The coder demonstrates a basic understanding of Java syntax and can achieve the intended functionality. However, the code lacks attention to detail regarding best practices for naming, formatting, and error handling. The reviewer points out several significant improvements related to readability and maintainability. While the coder might understand the concepts at a fundamental level, they haven't yet internalized the importance of writing clean, robust, and well-documented code. The ability to write concise and efficient code is also not fully demonstrated. The suggested improvements, particularly regarding naming and output formatting, are relatively straightforward and should be easily grasped by a coder with a higher level of proficiency.

## Iterations

1

## Code Comparison

< Manage app

## Project Settings

Select Programming Language

c|

C

Python

Java

C++

JavaScript

C#

Go

Other



# Code Review System using Generative AI

Paste Your Code Here

Start Review Process

Share



< Manage app

## Project Settings

Select Programming Language

C



# Code Review System using Generative AI

Paste Your Code Here

Start Review Process

Share



Rerun R

Settings

Print

Record a screencast

View all apps

About

Developer options

Clear cache C

< Manage app

# CONCLUSION

Our **AI-powered Code Review System** overcomes the limitations of existing systems by providing **more accurate, context-aware, and iterative feedback**. Unlike traditional AI models that struggle with deep architectural flaws and evolving best practices, our system integrates **Google Gemini AI, LangGraph, and Streamlit** to offer **automated yet adaptable** code analysis. It ensures **higher accuracy, reduced false positives, and efficient optimization**, making the review process **faster, smarter, and more reliable** while minimizing human effort.

**THANK YOU**